

11.1 An Introduction to Asymptotic Analysis

Previously, we have focused on how to save time *writing* the program. Now, we will learn how to make the best use of our computer's time and memory.

We can consider the process of writing efficient programs from two different perspectives:

1. Programming Cost (*everything in the course up to this date*)
 1. How long does it take for you to develop your programs?
 2. How easy is it to read or modify your code?
 3. How maintainable is your code? (very important — much of the cost comes from maintenance and scalability, not development!)
2. Execution Cost (*everything in the course from this point on*)
 1. **Time complexity**: How much time does it take for your program to execute?
 2. **Space complexity**: How much memory does your program require?

To give a sense of what is coming up, consider a **sorted** array. Our goal is to determine if there is a duplicate element in the list.

```
List<Integer> example = [-3, -1, 2, 4, 4, 8, 10, 12];
```

A **naïve algorithm** would be to compare every pair of elements. In the above example, we would compare -3 with every element in the list, then -1, then 2, etc.

A **better algorithm** would be to take advantage of the sorted nature of the list! Instead of comparing every pair of elements, we can compare each element with just the element next to it.

We can see that the **naïve algorithm** seems like it's doing a lot more unnecessary, redundant work than the **better algorithm**. But how much more work is it doing? How do we quantify how efficient a program is? This chapter will provide you the formal techniques and tools to compare the efficiency of various algorithms!

11.2 Runtime Characterization

We want to be able to *characterize* the runtime of the two algorithms we previously saw. In other words, we want to come up with some proxy that communicates the overall performance of each algorithm.

When characterizing runtimes, we have two goals in mind:

- They should be simple but mathematically rigorous.
- They should clearly demonstrate the superiority of one algorithm over another if one algorithm is better than the other.

```
{% embed url="https://www.youtube.com/watch?ab_channel=JoshHug&v=mRn8Z46psX8" %}
```

We have converted both the naïve algorithm and the better algorithm into Java code. The function `dup1` corresponds to the naïve algorithm and `dup2` corresponds to the better algorithm.

```
// Naïve algorithm: compare everything
public static boolean dup1(int[] A) {
    for (int i = 0; i < A.length; i += 1) {
        for (int j = i + 1; j < A.length; j += 1) {
            if (A[i] == A[j]) {
                return true;
            }
        }
    }
    return false;
}

// Better algorithm: compare only neighbors
public static boolean dup2(int[] A) {
    for (int i = 0; i < A.length - 1; i += 1) {
        if (A[i] == A[i + 1]) {
            return true;
        }
    }
    return false;
}
```

Technique 1

The first technique we will consider using for runtime characterization is directly measuring execution time in seconds using a client program.

There are a few ways we could do this:

- Use a physical stopwatch (not recommended).
- Use Unix's built in time command.
- Use the Princeton Standard library which has a stopwatch class.

Using any of these methods will show (with varying levels of accuracy, depending on whether the physical stopwatch path was chosen) that as input size increases, `dup1` takes a longer time to complete, whereas `dup2` completes at relatively around the same rate.

It seems like technique 1 works perfectly fine for characterizing our runtimes! It is very easy to understand the results of the experiment, and it is easy to implement. However, there are some serious cons associated it that dissuades us from using it for everything:

- It could take a *long* time to finish running.
- Running times can vary by machine, compiler, input data, etc.

For these reasons, technique 1 does not meet our goals in characterizing runtimes. It's simple, but it's not mathematically rigorous. Moreover, the differences based on machine, compiler, input, etc. mean that the results may not clearly demonstrate the relationship between dup1 and dup2.

{% embed url="https://www.youtube.com/watch?ab_channel=JoshHug&v=fxDIy6w09fw" %}

Technique 2A

Rather than physically timing the amount of time it takes for the algorithm to run, we can instead count the total number of operations completed by each algorithm! Intuitively, the algorithm that runs the fewer number of operations would be superior to the other. We can fix the input size N to be the same for both algorithms and compare the number of operations run.

Let us apply this to the dup1 algorithm with an input size of $N=10000$:

```
for (int i = 0; i < A.length; i += 1) {
  for (int j = i+1; j < A.length; j += 1) {
    if (A[i] == A[j]) {
      return true;
    }
  }
}
return false;
```

Operation	Count (for $N=10000$)
$i = 0$	1
$j = i+1$	1 (in the best case) to 10000 (in the worst case)
$<$	2 to 50,015,001
$+= 1$	0 to 50,005,000
$==$	1 to 49,995,000
array accesses	2 to 99,990,000

The operation $i = 0$ is only run a single time at the very start of the function call. $j = i+1$ is more complicated—in the best case when $A[0] == A[1]$, it only runs a single time (convince yourself of this fact!). In the worst case, j is initialized once for each value that i takes on in the outer loop, so it is initialized a total of 10000 times.

As we dive further into the loop, it gets progressively less feasible to calculate the exact counts. We will show later on that the exact numbers do not matter that much.

To summarize technique 2A, we have solved the issue of *machine independence*. Differences in machines do not affect the total of operations run (usually). However, it is tedious to compute all the counts for each operation. In addition, our chosen input size is arbitrary, and we do not know how much time it actually takes to run.

Technique 2B

We can resolve the issue of choosing input size by calculating the *symbolic count* instead, which means that we calculate our counts in terms of the input N . Using this technique, we can update our table to include the symbolic counts:

Operation	Symbolic Count	Count (for $N=10000$)
$i = 0$	1	1
$j = i+1$	1 to N	1 (in the best case) to 10000 (in the worst case)
$<$	2 to $(N^2 + 3N + 2)/2$	2 to 50,015,001
$+= 1$	0 to $(N^2 + N)/2$	0 to 50,005,000
$==$	1 to $(N^2 - N)/2$	1 to 49,995,000
array accesses	2 to $N^2 - N$	2 to 99,990,000

Using symbolic counts allows us to see how the algorithms *scales* with input size. However, it is now even more tedious to calculate! It also still does not tell the actual time.

11.3 Checkpoint: An Exercise

Exercise: Apply techniques 2A and 2B to `dup2`.

- Calculate the counts of each operation for the following code with respect to N .
- Predict the *rough* magnitudes of each one.

```
for (int i = 0; i < A.length - 1; i += 1){
    if (A[i] == A[i + 1]) {
        return true;
    }
}
return false;
```

{% embed url="https://www.youtube.com/watch?ab_channel=JoshHug&v=IxQh6SXRBRw" %}

Solution:

Note: It's okay if you were slightly off—as mentioned earlier, you want *rough* estimates.

Operation	Symbolic Count	Count (for $N=10000$)
$i = 0$	1	1

Operation	Symbolic Count	Count (for N=10000)
$j = i+1$	0 to N	0 to 10,000
$<$	0 to $N - 1$	0 to 9,999
$==$	1 to $N - 1$	1 to 9,999
array accesses	2 to $2N - 2$	2 to 19998

{% embed url="https://www.youtube.com/watch?ab_channel=JoshHug&v=jms0P6p6aQc" %} "I have another problem for you to solve (∞)..." - Josh Hug {% endembed %}

Let us compare the dup1 table with the dup2 table:

dup1 table:

Operation	Symbolic Count	Count (for N=10000)
$i = 0$	1	1
$j = i+1$	1 to N	1 (in the best case) to 10000 (in the worst case)
$<$	2 to $(N^2 + 3N + 2)/2$	2 to 50,015,001
$+= 1$	0 to $(N^2 + N)/2$	0 to 50,005,000
$==$	1 to $(N^2 - N)/2$	1 to 49,995,000
array accesses	2 to $N^2 - N$	2 to 99,990,000

dup2 table:

Operation	Symbolic Count	Count (for N=10000)
$i = 0$	1	1
$j = i+1$	0 to N	0 to 10,000
$<$	0 to $N - 1$	0 to 9,999
$==$	1 to $N - 1$	1 to 9,999
array accesses	2 to $2N - 2$	2 to 19998

We can see that dup2 performs significantly better than dup1 in the worst case!

One way to rationalize this is that it takes fewer operations for dup2 to accomplish the same goal as dup1.

A better realization is that the algorithm for dup2 scales much better in the worst case (e.g. $(N^2 + 3N + 2)/2$ vs N)

An even **better** realization is that parabolas (N^2) always grow faster than lines (N).

11.4 Asymptotic Behavior

In most applications, we are most concerned about what happens for very large values of N . This is known as the *asymptotic behavior*. We want to learn what types of algorithms are able to handle large amounts of data. Some examples of applications that require highly efficient algorithms are:

- Simulating the interactions of billions of particles
- Maintaining a social network with billions of users
- Encoding billions of bytes of video data

Algorithms that scale well have better *asymptotic* runtime behavior than algorithms that scale poorly.

```
{% embed url="https://www.youtube.com/watch?ab_channel=JoshHug&v=vxYADFsa3HU" %}
```

Let us return to the original problem of characterizing the runtimes of our `dup` functions. Recall that we desire characterizations that have the following features:

- Simple and mathematically rigorous.
- Clearly demonstrate the superiority of `dup2` over `dup1`.

We've accomplished the second task! We are able to clearly see that `dup2` performed better than `dup1`. However, we didn't do it in a very simple or mathematically rigorous way. Luckily, we can apply a series of simplifications to solve these issues.

Simplification 1: Consider Only the Worst Case

When comparing algorithms, we often only care about the worst case. The worst case is often where we see the most interesting effects, so we can usually ignore all other cases but the worst case.

Example:

Consider the operation counts of some algorithm below. What do you expect will be the order of growth of the runtime for the algorithm?

- N (linear)
- N^2 (quadratic)
- N^3 (cubic)
- N^6 (sextic)

Operation	Count
less than ($<$)	$100N^2 + 3N$

Operation	Count
greater than ($>$)	$N^3 + 1$
and ($\&\&$)	5000

Answer: N^3 (cubic)

Intuitively, N^3 grows faster than N^2 , so it would “dominate.” To help further convince you that this is the case, consider the following argument:

- Suppose the $<$ operator takes α nanoseconds, the $>$ operator takes β nanoseconds, and the $\&\&$ takes γ nanoseconds.
- The total time is $\alpha(100N^2 + 3) + \beta(2N^3 + 1) + 5000\gamma$ nanoseconds.
- For very large N , the $2\beta N^3$ term is much larger than others.
- It can help to think of it in terms of calculus. What happens as N approaches infinity? Which term ends up dominating?

Simplification 2: Restrict Attention to One Operation

Pick some representative operation to act as a proxy for overall runtime. From our dup example:

- Good choice: increment, or **less than** or **equals** or **array accesses**.
- Bad choice: **assignment of** $j = i + 1$, or $i = 0$.

The operation we choose is called the “**cost model**.”

Simplification 3: Eliminate Low Order Terms

Ignore lower order terms.

Sanity check: Why does this make sense? (Related to the checkpoint above!)

Simplification 4: Eliminate Multiplicative Constants

Ignore multiplicative constants.

- Why? No real meaning!
- By choosing a single representative operation, we already “threw away” some information.
- Some operations had counts of $3N^2$, $N^2/2$, etc. In general, they are all in the family/shape of N^2 .

Checkpoint Exercise:

Apply our four simplification rules to the dup2 table.

Operation	Symbolic Count
$i = 0$	1
$j = i+1$	0 to N
$<$	0 to $N - 1$
$==$	1 to $N - 1$
array accesses	2 to $2N - 2$

Example answer: array accesses with order of growth N .

$<$, $==$, and $j=i+1$ would be fine answers as well.

Simplification Summary

- Only consider the worst case.
- Pick a representative operation (aka: cost model)
- Ignore lower order terms
- Ignore multiplicative constants.

11.5 Simplified Analysis Process

Summary of our (Painful) Analysis Process

- Construct a table of exact counts of all possible operations (takes lots of effort!)
- Convert table into worst case order of growth using 4 simplifications.

We will now propose an alternative method that avoids building a table altogether!

{% embed url="https://www.youtube.com/watch?v=IJ1A8Jyeba0&ab_channel=JoshHug" %}

Our simplified analysis process will consist of:

- Choosing our cost model, which is the representative operation we want to count.
- Figuring out the order of growth for the count of our representative operation by either:
 - Making an exact count and discarding unnecessary pieces or...
 - Using intuition/inspection to determine orders of growth. This is something that comes with practice.

Example: Analysis of Nested For Loops - Exact Counts

Find the order of growth of the worst case runtime of `dup1`.

```
int N = A.length;
for (int i = 0; i < N; i += 1)
    for (int j = i + 1; j < N; j += 1)
        if (A[i] == A[j])
            return true;
return false;
```

We will choose our cost model to be the *number of == operations*.

Looking at the structure of the loops, the inner loop first gets run $j=N-1$ times. At the second iteration, $i=1$, so the inner loop runs an additional $j=N-2$ times. At the third iteration, $i=2$, so the inner loop runs an additional $j=N-3$ times. The total number of times the loop is run is thus:

$$\text{cost} = 1 + 2 + 3 + \dots + (N - 2) + (N - 1)$$

This cost can be simplified to $\frac{N(N-1)}{2}$ ([how?](#)). We can use simplification to throw away all lower order terms and constants to get the worst case order of growth N^2 .

Example: Analysis of Nested For Loops - Geometric Argument

- We can see that the number of equals can be given by the area of a right triangle, which has a side length of $N - 1$.
- Therefore, the order of growth of area is N^2 .
- This is definitely not something that is immediately obvious. It takes time and practice to see these patterns!

11.6 Summary

To summarize this chapter:

- Given a piece of code, we can express its runtime as a function $R(N)$
 - N is a **property** of the input of the function often representing the **size** of the input
- Rather than finding the exact value of $R(N)$, we only worry about finding the **order of growth** of $R(N)$.
- One approach (not universal):
 - Choose a representative operation
 - Let $C(N)$ be the count of how many times that operation occurs as a function of N .
 - Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
 - If operation takes constant time, then $R(N) \in \Theta(f(N))$.

11.7 Exercises

Factual

1. Analyze the runtime of the following code in terms of N :

```
for (int i = 0; i < N; i++) {  
    int j = 0;  
    while (j < N) {  
        j = N;  
    }  
}
```

2. Let $f(N) = 2N$. Which of the following statements is true?

- ☐ $f(N) \in \Theta(1)$
- ☐ $f(N) \in \Theta(N)$
- ☐ $f(N) \in \Theta(N^2)$
- ☐ $f(N) \in O(1)$
- ☐ $f(N) \in O(N)$
- ☐ $f(N) \in O(N^2)$

- Problem 1
- Problem 2

Procedural

1. **True or false.** Suppose we have a function f , and we are told $f(N) \in \Theta(N^2)$. If we run f on an input of size N , then an input of size $2N$, it will take roughly 4 times as long.
2. **True or false.** Suppose we have a function f , and we are told $f(N) \in \Theta(N^2)$. If we run f on an input of size 100, then an input of size 200, it will take roughly 4 times as long.

- Problem 1
- Problem 2

Metacognitive

1. Why do we use asymptotics instead of empirical timing (for example, like the Stopwatch class from Lab 3)?

► Problem 1

11. Asymptotics I